DevSecOps Course Outline

Part 1: DevOps roadmap with security built into every phase | Part 2: 6-week DevSecOps course

Course principles

- **Foundations first.** Security skills sit on top of solid Linux, cloud and Git knowledge.
- **Concepts over tools.** Tools change; threat models and secure design principles last.
- **Shift left.** Find and fix security issues early, in code and pipelines, not after release.
- **Projects before certifications.** Secure a real system first, then certify to prove it.
- **Combine, do not isolate.** Add each security control to the same growing project.
- **Automate security.** A control that is not in the pipeline will eventually be skipped.

Contents

Part	What it covers
1	DevOps roadmap (6 phases, 6 to 10 months) with a security layer added to each phase
2	Focused 6-week DevSecOps course: weekly goals, labs and deliverables, plus assessment and tools

Part 1: The DevOps Roadmap, Secured

Phase	Topic	Beginner	Some experience
1	Foundations: Linux, Bash, Git	2 months	1 month
2	Cloud basics (AWS)	2 months	1 month
3	Infrastructure as Code	1 month	1 month
4	Containers and orchestration	2 months	1 month
5	CI/CD	2 months	1 month
6	Observability	1 month	1 month
Total		**10 months**	**6 months**

Phase 1: Foundations (Linux, Bash, Git)

Why: You will manage Linux-powered systems for the rest of your career, so this base must be solid.

Modules

1. Linux fundamentals: filesystem, users and permissions, processes, package management, networking
2. Bash scripting: variables, conditionals, loops, functions, automation
3. Git: init, commit, branching, merging, conflicts, GitHub workflows

Security layer: Linux hardening (least-privilege permissions, SSH keys, firewall basics); secrets never committed to Git; branch protection and signed commits.

Security tools: Gitleaks, pre-commit hooks, ufw/iptables

Hands-on: Write scripts that automate a real task and keep them in a Git repository.

Milestone: Comfortable in the terminal, with a versioned scripts repo.

Phase 2: Cloud Basics (AWS)

Why: Compute, storage and networking work the same on any cloud. Learn concepts first, then how AWS implements them.

Modules

1. Cloud concepts: compute, storage, networking
2. AWS core services: a small set is enough
3. Identity and access basics, and cost awareness

Security layer: Shared responsibility model, IAM least privilege, security groups, encryption at rest and in transit, audit logging.

Security tools: AWS IAM, Security Groups, KMS, CloudTrail

Hands-on: Deploy a simple app manually on AWS infrastructure.

Milestone: A running app on cloud infrastructure that you built by hand.

Note: Do not chase certifications here. Certify only after building real projects.

Phase 3: Infrastructure as Code

Why: Learn what IaC is, why it exists and what problems it solves, then the tool (Terraform or Pulumi).

Modules

1. IaC concepts: declarative vs manual infrastructure, repeatability, version control
2. Terraform on AWS: providers, resources, variables, state, modules
3. Automating repetitive tasks and different parts of the system

Security layer: Scan IaC for misconfigurations, protect Terraform state, keep credentials out of code, enforce rules with policy as code.

Security tools: Checkov, tfsec, OPA, HashiCorp Vault

Hands-on: Recreate your Phase 2 manual setup entirely in Terraform code.

Milestone: Infrastructure that can be created and destroyed from code.

Phase 4: Containers and Orchestration

Why: The first of the two most rewarding phases. Master the basics, not every feature.

Modules

1. Container concepts: how containers work and why they matter
2. Docker: images, Dockerfiles, volumes, networking, registries
3. Kubernetes: pods, deployments, services, config and secrets

Security layer: Minimal base images, non-root containers, image scanning and signing, Kubernetes RBAC, network policies and pod security.

Security tools: Trivy, Cosign, Kubernetes RBAC and NetworkPolicy

Hands-on: Containerize your app, then run it on Kubernetes.

Milestone: App running in containers, orchestrated by Kubernetes.

Phase 5: CI/CD (Automated Release and Deployment)

Why: CI/CD is the backbone of DevOps and of DevSecOps. Pick one tool: Jenkins, GitHub Actions or GitLab CI.

Modules

1. CI/CD concepts: build, test, deploy, environments, release strategies
2. Pipeline tooling: pick one tool and go deep
3. Connecting the pipeline to containers, Kubernetes or AWS

Security layer: Security gates in the pipeline: secret scan, SAST, dependency scan (SCA), container scan, DAST, SBOM; builds fail on high severity.

Security tools: Semgrep, Dependabot, Trivy, OWASP ZAP, Syft

Hands-on: Build a pipeline end to end that takes a code commit through to a containerized deployment.

Milestone: Push code, and it deploys automatically.

Phase 6: Observability

Why: The final, more advanced step. Understand the fundamentals and how to implement them.

Modules

1. Observability concepts: metrics, monitoring, alerting
2. Prometheus and Grafana: collecting metrics and building dashboards
3. Applying monitoring to your deployed system

Security layer: Security monitoring, runtime threat detection, alerting on suspicious activity, incident response basics.

Security tools: Prometheus, Grafana, Falco, audit logs

Hands-on: Add monitoring and dashboards to your running application.

Milestone: A fully monitored, automated system.

Running project (ties it all together)

One application evolves through every phase:

Phase	What students add	Security addition
1	Scripts and a Git repository	Secret scanning, protected branches
2	Manual deployment on AWS	Least-privilege IAM, encryption
3	Infrastructure rebuilt in Terraform	IaC scanning, safe state
4	Dockerized and running on Kubernetes	Image scans, RBAC, network policies
5	Automated CI/CD pipeline	SAST, SCA, DAST, SBOM gates
6	Prometheus and Grafana monitoring	Runtime detection and alerts

Part 2: 6-Week DevSecOps Course

Level: Beginner to intermediate. **Prerequisites:** basic Git and command-line skills. **Format:** one 2-hour session per week (30 min concept, 90 min lab) plus a weekly deliverable. **Running project:** students secure a small web app across all six weeks.

Week 1: Foundations and Secure Source Control

Goal: Understand why security moves left and lock down the repository.

- DevOps vs DevSecOps, shift-left security, the CIA triad
- The secure SDLC and common attack surfaces
- Git security: branch protection, signed commits, code review rules
- Secrets leakage and how to prevent it

Lab: Enable branch protection on the app repo and add a Gitleaks pre-commit hook.

Deliverable: Repo with protected main and a passing secret scan.

Week 2: Secure Coding and CI Pipelines

Goal: Automate security checks on every commit.

- CI/CD concepts and anatomy of a pipeline
- OWASP Top 10 (injection, broken access control, XSS)
- Static Application Security Testing (SAST)
- Failing builds on severity thresholds

Lab: Build a GitHub Actions pipeline with linting and Semgrep, and fix three planted vulnerabilities.

Deliverable: Pipeline that fails on high-severity findings.

Week 3: Dependency and Container Security

Goal: Secure what you build on and what you ship.

- Software Composition Analysis (SCA) and vulnerable dependencies
- Supply-chain attacks and Software Bills of Materials (SBOM)
- Docker hardening: minimal base images, non-root users, multi-stage builds
- Image scanning and signing

Lab: Enable Dependabot, containerize the app, scan with Trivy and generate an SBOM.

Deliverable: Hardened Dockerfile with a clean scan report.

Week 4: Infrastructure as Code and Secrets Management

Goal: Treat infrastructure like code, and secure it the same way.

- Infrastructure as Code (Terraform basics)
- Cloud misconfigurations and the principle of least privilege
- Policy as code (OPA)
- Secrets management: vaults, environment variables, rotation

Lab: Scan Terraform with Checkov, fix misconfigurations, and move credentials into a secrets manager (Vault or GitHub Secrets).

Deliverable: Secure IaC template and a secrets-handling write-up.

Week 5: Dynamic Testing, Deployment and Runtime Security

Goal: Test the running app and detect attacks in production.

- Dynamic Application Security Testing (DAST)
- Secure deployment strategies (blue-green, canary, rollbacks)
- Kubernetes security basics: RBAC, network policies, pod security
- Logging, monitoring, alerting and incident response

Lab: Run OWASP ZAP against staging, then set up basic monitoring and alerts (Prometheus and Grafana, or Falco).

Deliverable: DAST report and a one-page incident response checklist.

Week 6: Governance, Metrics and Capstone

Goal: Tie everything together and present a complete secure pipeline.

- Threat modeling (STRIDE)
- Compliance as code and audit trails (ISO 27001, SOC 2, NIST overview)
- Metrics: mean time to remediate, vulnerability density, DORA metrics
- Building a security culture and security champions

Lab: Capstone: present an end-to-end pipeline with secret scanning, SAST, SCA, container and IaC scanning, DAST and monitoring, plus a threat model and remediation report.

Deliverable: Capstone presentation and report.

Assessment

Component	Weight
Weekly labs and deliverables	40%
Short weekly quizzes	15%
Capstone project and presentation	35%
Participation and peer review	10%

Tool stack at a glance

Stage	Tools
Source and secrets	Git, GitHub, Gitleaks
Build and CI	GitHub Actions
SAST / SCA	Semgrep, Dependabot, Trivy
Containers	Docker, Trivy, Cosign
IaC and policy	Terraform, Checkov, OPA, Vault
DAST and runtime	OWASP ZAP, Prometheus, Grafana, Falco

Common mistakes to avoid

- Treating security as a final checkpoint instead of building it into every phase
- Learning tools in isolation without combining them into one pipeline
- Running scanners but never fixing or triaging the findings
- Collecting shallow courses and certificates instead of securing a real system